

One Bite Restaurant Management System

CONTENTS

- Program Overview.
- Folder Structure.
- Functional Requirements.
- Data Management.
- Key Classes and Methods.
- Usage Workflow.
- Conclusion.

Authors:

Nisha Modi

Vimal Shahi

Gaurav Karki

Kumkum Kapkoti

Anjali Nagarkoti

(Team 4, Batch 1, Full-Stack Web Development, Indixpert)

Program Overview:

The **One Bite Restaurant Management System** is a Python application that streamlines restaurant operations, including user authentication, table booking, menu management, order processing, and billing. It allows admins and staff to manage roles, handle reservations, update menus, process orders, and generate bills with payment handling. Using JSON files for data storage, it operates efficiently with a database, enhancing restaurant management and customer service.

Folder Structure:

The system is organized into a modular structure:

- `src/authentication` : Handles user authentication and management.
- `src/booking` : Manages table bookings with time slots.
- `src/dashboard` : Provides all the dashboards.
- `src/database` : Stores data in JSON files for users, bookings, orders, etc.
- `src/menu` : Manages menu items and prices.
- `src/orders` : Processes customer orders and links them to bookings.
- `src/manage_bill` : Handles bill generation and payment processing.
- `src/payments` : Handles payment transactions.
- `src/utility` : Contains utilities for validation, logging, and color-coded output
- `main.py` : Entry point for the application.

Functional Requirements:

- **User Authentication** : Admin and staff can log in and sign up. Admins can manage staff users.
- **Table Booking** : Staff can book, view, and cancel table reservations for customers.
- **Menu Management** : Admins can add, update, or remove items from the menu. Staff can view the menu.
- **Order Management** : Staff can create, update, and cancel orders for dine-in or takeout.
- **Billing** : Generate bills, process payments, and save transaction details.
- **Logging** : Logs user and admin activities.
- **Role-Based Access Control** : Admins have elevated permissions compared to staff, ensuring sensitive operations are restricted.

Data Management:

The system uses JSON files for data storage, that are stored in the database folder.

- **user.json** : Stores user credentials and roles.
- **booking.json** : Stores table booking information.
- **menu.json** : Stores menu items with prices.
- **order.json** : Stores customer orders and details.
- **bill.json** : Stores billing records.
- **payment.json** : Stores payment details.

Each file is loaded into the application at runtime and saved upon updates, ensuring persistent data storage.

Key Classes and Methods:

- **System (user_authentication.py)** : Manages user login, sign-up, and user management.
- **TableBookingSystem (table_booking.py)**: Handles viewing, booking, and canceling tables.
- **OrderFeature (order_feature.py)**: Manages orders, including adding, updating, and canceling.
- **BillFeature (bill_feature.py)**: Handles bill generation, updates, and payment processing.
- **Menu (menu.py)**: Loads, views, and manages menu items.

Usage Workflow:

- Start the application using main.py.
- Login or Signup as admin or staff.
- Admins can manage users, update the menu, and oversee orders.
- Staff can handle bookings, place orders, and generate bills.
- Process payments for orders and generate billing reports.
- Log out to complete the session.

Once logged in, users are presented with dashboards tailored to their role. Admins can perform advanced functions such as user management, while staff focus on daily tasks like booking tables and managing orders. The system logs all user activities, which can be reviewed in the “activity.log” file.

Conclusion:

In conclusion, The Restaurant Management System is a robust solution for managing restaurant operations efficiently. By using a file-based data storage approach, it eliminates the need for a database server, making it lightweight and easy to deploy.

By leveraging a modular design with JSON-based data storage, the system is lightweight, efficient, and easily deployable. It empowers restaurant administrators and staff to handle their tasks more effectively, reducing manual effort and enhancing overall service quality.